

Pontificia Universidad Católica de Chile
Departamento de Ciencia de la Computación
IIC2613 - Inteligencia Artificial

Tarea 5

5 de julio de 2026

1º semestre 2026

Rafael Lorca - 23641789

DCC TiCTacToe Supreme

Actividad 1: Poda Alpha-Beta

Para todos los experimentos se utilizó la función de evaluación **strategic_eval**.

Cuadro 1: Resultados de los Juegos sin poda. Profundidad = 1

Juego	**Resultado**	**AI 1**	**AI 2**
Juego 1	AI 1	0.0028s	0.0039s
Juego 2	AI 2	0.0034s	0.0024s
Juego 3	AI 1	0.004s	0.0019s
Juego 4	AI 1	0.0058s	0.003s
Juego 5	AI 1	0.0047s	0.0019s

Cuadro 2: Resultados de los Juegos con poda. Profundidad = 1

Juego	**Resultado**	**AI 1**	**AI 2**
Juego 1	AI 1	0.0038s	0.003s
Juego 2	AI 1	0.003s	0.003s
Juego 3	AI 1	0.0022s	0.0029s
Juego 4	AI 2	0.0033s	0.002s
Juego 5	AI 2	0.005s	0.0032s

Cuadro 3: Resultados de los Juegos sin poda. Profundidad = 2

Juego	**Resultado**	**AI 1**	**AI 2**
Juego 1	AI 2	0.0147s	0.0083s
Juego 2	AI 2	0.0134s	0.0085s
Juego 3	AI 1	0.0114s	0.0074s
Juego 4	AI 2	0.0132s	0.0085s
Juego 5	AI 1	0.0113s	0.0082s

Cuadro 4: Resultados de los Juegos con poda. Profundidad = 2

Juego	**Resultado**	**AI 1**	**AI 2**
Juego 1	AI 1	0.0056s	0.0044s
Juego 2	AI 1	0.0063s	0.0051s
Juego 3	AI 1	0.0061s	0.0049s
Juego 4	AI 1	0.0052s	0.0047s
Juego 5	AI 1	0.0065s	0.005s

Cuadro 5: Resultados de los Juegos sin poda. Profundidad = 3

Juego	**Resultado**	**AI 1**	**AI 2**
Juego 1	AI 2	0.161s	0.1416s
Juego 2	AI 2	0.2191s	0.1497s
Juego 3	AI 1	0.1931s	0.1191s
Juego 4	AI 2	0.2021s	0.1011s
Juego 5	AI 2	0.1875s	0.0921s

Cuadro 6: Resultados de los Juegos con poda. Profundidad = 3

Juego	**Resultado**	**AI 1**	**AI 2**
Juego 1	AI 2	0.05s	0.0443s
Juego 2	AI 1	0.0384s	0.0408s
Juego 3	AI 1	0.0456s	0.0502s
Juego 4	AI 2	0.0514s	0.0453s
Juego 5	AI 2	0.0443s	0.0355s

Cuadro 7: Resultados de los Juegos. Profundidad = 4

Juego	**Resultado**	**AI 1**	**AI 2**
Sin Poda	AI 1	0.8347s	0.5678s
Con Poda	AI 1	0.1454s	0.1183s

Analizemos todas las tablas. El cuadro 1 muestra los resultados de los juegos sin poda con profundidad 1, el cuadro 2 muestra para la misma profundidad pero con poda. En este caso que la profundidad del árbol al ser 1, solamente puede expandir 1 nodo, luego no hay mucha diferencia entre correr el algoritmo Minimax con y sin la poda alpha-beta. Los tiemmpos de evaluación promedio para los experimentos distintos son bastante similares, eso sí, hay que considerar que siempre el primer movimiento que realiza el jugador AI 1 requiere un poco más de computación, luego es el que se demora más y es por eso que estos tiempos son mayores que los del jugador 2. Claramente vemos lo que propone la poda alpha-beta: en el peor de los casos, es igual a Minimax sin la poda, sin embargo en el mejor de los casos, acortaría el tiempo a la mitad [1], [2]:

Complejidad de algoritmo MiniMax: $O(b^m)$

Óptimo con poda alpha-beta: $O(b^{m/2})$

Donde $m =$ es la profundidad máxima y $b =$ factor de ramificación.

Para el resto de profundidades ya se nota lo anterior, con poda el tiempo promedio se acorta a la mitad en casi todos los juegos. Por ejemplo con profundidad 3, los resultados de los tiempos de juego sin poda son de al rededor de 0.185 segundos para la AI 1. mientras que con poda es de 0.04 segundos. Para el experimento con profundidad 4, claramente se demoró bastante en correr, pero se ve un resultado extraordinario, el tiempo para la IA 1 sin poda es 5.7 veces más que con poda y para la IA 2 el tiempo sin poda es 4.8 veces más alto que el con poda.

Actividad 2: Comparación de profundidades

Cuadro 8: Experimento 2.1

Juego	Ganador	Eval P1	D1	D2	Avg T P1	Avg T P2
1	AI 1	evaluate	1	1	0.002s	0.002s
2	AI 2	evaluate	1	1	0.001s	0.001s
3	AI 1	evaluate	1	1	0.001s	0.001s
4	AI 1	evaluate	1	1	0.001s	0.001s
5	AI 1	evaluate	1	1	0.001s	0.001s
6	AI 1	evaluate	1	1	0.001s	0.001s
7	AI 1	evaluate	1	1	0.002s	0.002s
8	AI 2	evaluate	1	1	0.002s	0.001s
9	AI 2	evaluate	1	1	0.001s	0.001s
10	AI 1	evaluate	1	1	0.002s	0.001s

Cuadro 9: Experimento 2.2

Juego	Ganador	Eval P1	D1	D2	Avg T P1	Avg T P2
1	AI 1	evaluate	1	2	0.001s	0.003s
2	AI 2	evaluate	1	2	0.001s	0.003s
3	AI 1	evaluate	1	2	0.001s	0.003s
4	AI 1	evaluate	1	2	0.001s	0.002s
5	AI 2	evaluate	1	2	0.001s	0.002s
6	AI 2	evaluate	1	2	0.001s	0.003s
7	AI 2	evaluate	1	2	0.001s	0.002s
8	AI 2	evaluate	1	2	0.001s	0.002s
9	AI 2	evaluate	1	2	0.001s	0.002s
10	AI 2	evaluate	1	2	0.001s	0.003s

Cuadro 10: Experimento 2.3

Juego	Ganador	Eval P1	D1	D2	Avg T P1	Avg T P2
1	AI 2	evaluate	2	1	0.003s	0.001s
2	AI 1	evaluate	2	1	0.003s	0.001s
3	AI 1	evaluate	2	1	0.003s	0.001s
4	AI 1	evaluate	2	1	0.003s	0.001s
5	AI 1	evaluate	2	1	0.003s	0.001s
6	AI 2	evaluate	2	1	0.004s	0.001s
7	AI 1	evaluate	2	1	0.003s	0.001s
8	AI 1	evaluate	2	1	0.003s	0.001s
9	AI 1	evaluate	2	1	0.004s	0.001s
10	AI 1	evaluate	2	1	0.004s	0.001s

Cuadro 11: Experimento 2.4

Juego	Ganador	Eval P1	D1	D2	Avg T P1	Avg T P2
1	AI 2	evaluate	1	3	0.001s	0.014s
2	AI 1	evaluate	1	3	0.001s	0.016s
3	AI 2	evaluate	1	3	0.001s	0.016s
4	AI 2	evaluate	1	3	0.001s	0.017s
5	AI 2	evaluate	1	3	0.001s	0.013s
6	AI 1	evaluate	1	3	0.001s	0.014s
7	AI 2	evaluate	1	3	0.001s	0.018s
8	AI 1	evaluate	1	3	0.001s	0.014s
9	AI 2	evaluate	1	3	0.001s	0.015s
10	AI 1	evaluate	1	3	0.001s	0.008s

Cuadro 12: Experimento 2.5

Juego	Ganador	Eval P1	D1	D2	Avg T P1	Avg T P2
1	AI 1	evaluate	3	1	0.020s	0.001s
2	AI 2	evaluate	3	1	0.020s	0.001s
3	AI 1	evaluate	3	1	0.019s	0.001s
4	AI 1	evaluate	3	1	0.019s	0.001s
5	AI 2	evaluate	3	1	0.016s	0.001s
6	AI 1	evaluate	3	1	0.016s	0.001s
7	AI 2	evaluate	3	1	0.010s	0.001s
8	AI 1	evaluate	3	1	0.015s	0.001s
9	AI 1	evaluate	3	1	0.013s	0.001s
10	AI 1	evaluate	3	1	0.022s	0.001s

Claramente para todos los experimentos realizados se puede apreciar que hay una correlación directa entre el tiempo promedio de movimiento con la profundidad máxima de cada árbol Minimax asociado a cada jugador, se puede apreciar de mejor manera en los cuadros 11 y 12 en dónde los jugadores con depth 3 presentan tiempos promedio de 0.008 para el player 2 con depth 3 versus 0.001 para el player 1 con depth 1.

En el caso del número de victorias por experimento, se ve tambien una correlación entre la profundidad a la hora de determinar que agente sale victorioso. Similar a lo anterior, mientras más profundidad, más probabilidades de ganar. Haciendo alusión al funcionamiento de Minimax: Cada jugador tiene su propio árbol, luego mientras más expansiones se tengan y más profundo el árbol, hay más chances de obtener buenos valores para cada estado del juego posible. Pero no solo lo anterior, sino que además el jugador con más profundidad puede anticipar la respuesta de su contrincante, por ejemplo teniendo una profundidad de 2: en la primera capa maximizas el valor del movimiento elegido (MAX), como tu profundidad es de dos, puedes expandir nodos y anticipar la jugada del contrincante en la segunda capa (MIN). En este caso y como dice el dicho popular: el conocimiento es poder, luego el jugador con mayor conocimiento, no solo de sus posibles jugadas, sino que también las de su rival, gana.

Actividad 3: Comparación de funciones de evaluación

Cuadro 13: Experimento 3.1

Juego	Ganador	Eval P1	Eval P2	D1	D2	Avg T P1	Avg T P2
1	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s
2	AI 1	evaluate	strategic_eval	1	1	0.001s	0.001s
3	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s
4	AI 1	evaluate	strategic_eval	1	1	0.001s	0.001s
5	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s
6	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s
7	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s
8	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s
9	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s
10	AI 2	evaluate	strategic_eval	1	1	0.001s	0.001s

Cuadro 14: Experimento 3.2

Juego	Ganador	Eval P1	Eval P2	D1	D2	Avg T P1	Avg T P2
1	AI 1	strategic_eval	evaluate	1	1	0.001s	0.001s
2	AI 1	strategic_eval	evaluate	1	1	0.001s	0.001s
3	AI 1	strategic_eval	evaluate	1	1	0.002s	0.001s
4	AI 1	strategic_eval	evaluate	1	1	0.001s	0.001s
5	AI 2	strategic_eval	evaluate	1	1	0.001s	0.001s
6	AI 1	strategic_eval	evaluate	1	1	0.001s	0.001s
7	AI 1	strategic_eval	evaluate	1	1	0.001s	0.001s
8	AI 1	strategic_eval	evaluate	1	1	0.001s	0.000s
9	AI 1	strategic_eval	evaluate	1	1	0.002s	0.001s
10	AI 2	strategic_eval	evaluate	1	1	0.002s	0.001s

Cuadro 15: Experimento 3.3

Juego	Ganador	Eval P1	Eval P2	D1	D2	Avg T P1	Avg T P2
1	AI 2	evaluate	strategic_eval	1	3	0.001s	0.015s
2	AI 2	evaluate	strategic_eval	1	3	0.000s	0.019s
3	AI 2	evaluate	strategic_eval	1	3	0.001s	0.015s
4	AI 2	evaluate	strategic_eval	1	3	0.000s	0.021s
5	AI 2	evaluate	strategic_eval	1	3	0.000s	0.012s
6	AI 2	evaluate	strategic_eval	1	3	0.001s	0.029s
7	AI 2	evaluate	strategic_eval	1	3	0.001s	0.022s
8	AI 2	evaluate	strategic_eval	1	3	0.000s	0.024s
9	AI 2	evaluate	strategic_eval	1	3	0.001s	0.024s
10	AI 2	evaluate	strategic_eval	1	3	0.001s	0.024s

Cuadro 16: Experimento 4.4

Juego	Ganador	Eval P1	Eval P2	D1	D2	Avg T P1	Avg T P2
1	AI 1	evaluate	strategic_eval	3	1	0.011s	0.001s
2	AI 1	evaluate	strategic_eval	3	1	0.016s	0.001s
3	AI 2	evaluate	strategic_eval	3	1	0.009s	0.001s
4	AI 2	evaluate	strategic_eval	3	1	0.010s	0.001s
5	AI 1	evaluate	strategic_eval	3	1	0.011s	0.001s
6	AI 1	evaluate	strategic_eval	3	1	0.011s	0.001s
7	AI 1	evaluate	strategic_eval	3	1	0.011s	0.001s
8	AI 1	evaluate	strategic_eval	3	1	0.012s	0.001s
9	AI 2	evaluate	strategic_eval	3	1	0.012s	0.001s
10	AI 2	evaluate	strategic_eval	3	1	0.008s	0.001s

Los experimentos anteriores nuevamente nos muestran un resultado claro, la importancia de la función de evaluación. Cuando se tiene la misma profundidad, gana muchas más veces el agente con la función de evaluación estratégica, esto debido a que toma muchas más variables en cuenta y pondera los mini tableros según su posición y si es que está alineado con otro que se ha ganado en la proximidad, además del riesgo de robo y la penalización de sub tableros casi llenos. Los otros dos experimentos sirven para ver que pesa más: la profundidad o la función de evaluación. Claramente cuando se tiene una función de evaluación más sofisticada y más profundidad, ese agente domina por completo al otro (Experimento 3.3), pero cuando un agente presenta más profundidad y el otro una función de evaluación más sofisticada (al menos para el experimento 4.4), notamos que es casi lo mismo, aunque está un poco por encima el agente que presenta mayor profundidad, en este experimento ganó el agente 1 6/10 partidas, con la función normal de evaluacioń pero con una profundidad de 3. Igualmente, con los resultados de estos últimos dos experimentos, puedo decir que una mejor funciónd e búsqueda si puede compensar una menor profundidad de búsqueda, a lo mejor hay una función mejor que la planteada que puede contrarrestar esta diferencia.

Actividad 4: Nueva función de evaluación

En general, la nueva función que armé se parece mucho a la función de strategic eval, solamente le asigna pesos a las diversas señales que strategic eval considera y añade una nueva: el número de amenazas dobles, ya que en strategic eval se tiene las lineas:

```
if mini.winner == my_mark:
        my_boards += 1
    elif mini.winner == opp_mark:
        opp_boards += 1
    elif not mini.is_done():
            my_mini_threats += _count_two_in_a_row(mini.cells, my_mark)
            opp_mini_threats += _count_two_in_a_row(mini.cells, opp_mark)
            my_mini_ones += _count_one_in_a_row(mini.cells, my_mark)
            opp_mini_ones += _count_one_in_a_row(mini.cells, opp_mark)
```

En la mia se tiene:

```
opp_mini_threats += opp_t
            my_mini_ones += _count_one_in_a_row(mini.cells, my_mark)
            opp_mini_ones += _count_one_in_a_row(mini.cells, opp_mark)
            # Forks en los mini tableros
            if my_t >= 2:
                my_mini_forks += 1
            if opp_t >= 2:
                opp_mini_forks += 1

# ...

# Agregamos los forks nomas encima de lo que strategic eval estaba
# haciendo bien
    mini_fork_score = 1.5 * (my_mini_forks - opp_mini_forks)
    meta_fork_score = 10.0 * (my_meta_fork  - opp_meta_fork)
```

Como bien sale en el código comentado, se cuenta no solo la existencia de una amenaza (strategic), sino cuando hay presente 2 o más amenazas ya que en el tablero no se puede defender de dos amenazas a la vez, por lo que además de contar las amenazas, se cuentan los forks (se hace lo mismo en el meta tablero). Creo que es buena para el problema ya que primero, detecta una nueva señal que serían los forks (tener un fork en el meta-tablero casi significa que ganaste). Vamos a ver el rendimiento de 2 agentes con la misma profundidad donde parte el agente con mi función custom:

Juego	Ganador	Eval P1	Eval P2	D1	D2	Avg T P1	Avg T P2
1	AI 2	custom_eval	strategic_eval	3	3	0.021s	0.014s
2	AI 2	custom_eval	strategic_eval	3	3	0.018s	0.012s
3	AI 1	custom_eval	strategic_eval	3	3	0.017s	0.013s
4	AI 1	custom_eval	strategic_eval	3	3	0.015s	0.012s
5	AI 1	custom_eval	strategic_eval	3	3	0.017s	0.017s
6	AI 1	custom_eval	strategic_eval	3	3	0.018s	0.013s
7	AI 1	custom_eval	strategic_eval	3	3	0.016s	0.013s
8	AI 1	custom_eval	strategic_eval	3	3	0.018s	0.014s
9	AI 1	custom_eval	strategic_eval	3	3	0.018s	0.014s
10	AI 2	custom_eval	strategic_eval	3	3	0.020s	0.012s

Como se puede ver en este cuadro final el agente 1 gana 7/10 de los juegos anteriores, dominando completamente al otro agente con strategic eval. También se aprecia un mayor tiempo de cómputo. Creo que es entendible esta mejora en el rendimiento ya que el código de la función custom es basicamente la evaluación estratégica con información adicional.

Referencias

[1] Russell, S. J., & Norvig, P. (2004). Inteligencia artificial: Un enfoque moderno (2.ª ed.). Pearson Educación. https://luismejias21.wordpress.com/wp-content/uploads/2017/09/inteligencia-artificial-un-enfoque-moderno-stuart-j-russell.pdf"

[2] GeeksforGeeks. (16 de enero de 2023). Minimax algorithm in game theory — Set 4 (Alpha-beta pruning). https://www.geeksforgeeks.org/dsa/minimax-algorithm-in-game-theory-se

Uso de IA

Se utilizó IA para la generación de una parte del código en el archivo main. Le pedí a la IA que me hiciera un código para que pueda copiar y pegar las tablas en latex directamente desde la terminal a mi archivo. Se utilizó también IA como guia para entender algunas funciones y correr simulaciones con animaciones del código minimax.